

ASSIGNMENT

CSA5116 – CRYPTOGRAPHY AND NETWORK SECURITY

Covering CO3, CO4, CO5 (Unit III–V)

A. Assignment Information

Department	Information Technology
Programme	B.E. / B. Tech – IT
Course Code & Course Name	CSA5116 – Cryptography and Network Security
Academic Year / Batch	2026 – 2027
Faculty Name	Dr. E. Anbalagan
Assignment Title	Design and Evaluation of a Secure Symmetric–Asymmetric Encryption Framework for Enterprise Networks
Date of Issue	01.09.2026
Date of Submission	02.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO3, CO4, CO5 – Design, analyse and evaluate cryptographic mechanisms for secure enterprise communication
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Hybrid encryption underpins secure enterprise messaging, banking, healthcare data exchange and e-government services worldwide.

B. Assignment Problem / Challenge

This assignment requires students to apply core Cryptography and Network Security concepts to a realistic enterprise communication problem. Students must analyse the given scenario, compare symmetric and asymmetric encryption mechanisms, design a hybrid encryption framework, implement a working prototype, evaluate its performance and security properties, and justify the most suitable cryptographic strategy for protecting enterprise data.

C. Problem Statement

Scenario: Design and Evaluation of a Secure Symmetric–Asymmetric Encryption Framework for Enterprise Networks

Modern enterprise networks exchange large volumes of sensitive information such as customer records, financial transactions, employee data, business documents and confidential communications. The organisation faces a fundamental architectural challenge: symmetric encryption is efficient for bulk data but creates key-distribution and key-management difficulties, while asymmetric encryption supports secure key distribution but introduces substantially greater computational overhead for large data volumes.

As part of the security engineering team, you are responsible for:

- Designing a hybrid encryption architecture that uses AES for bulk data encryption and RSA for session-key protection and exchange.
- Generating the cryptographic keys required for the communication process.
- Encrypting representative plaintext using the symmetric session key and protecting that session key using the recipient's RSA public key.
- Demonstrating secure transfer, session-key recovery and plaintext decryption.
- Analysing key generation, encryption, decryption, computational overhead and key-management requirements.
- Comparing the hybrid approach with symmetric-only and asymmetric-only alternatives and justifying the final architectural choice.

D. Requirements and Constraints

Functional Requirements

- The system shall accept a plaintext message or representative confidential document.
- It shall generate or establish a symmetric session key for each communication session.
- It shall encrypt the plaintext using the symmetric mechanism (AES).
- It shall protect the session key using the recipient's asymmetric public key (RSA).
- It shall transfer the encrypted data and protected session key as a transferable package.
- It shall recover the session key using the authorized recipient's private key and decrypt the ciphertext.
- It shall verify that the recovered plaintext matches the original plaintext.
- It shall support test cases for valid and invalid decryption conditions.

Non-Functional Requirements

Requirement	Description
Security	Keys and plaintext should be handled only for the required operation and not unnecessarily exposed.
Confidentiality	Intercepted ciphertext should not directly reveal the original message.
Performance	Bulk data encryption should be handled by the efficient symmetric mechanism.
Scalability	The architecture should suit multiple users and distributed offices with proper key management.
Reliability	Correct key use should reproduce the original plaintext without alteration.
Maintainability	The system should separate data encryption from session-key protection.
Usability	The workflow should be understandable and repeatable for authorized users.
Auditability	Important security operations should be observable through controlled logs or output messages.

Technical Constraints

- Symmetric algorithm: AES (key size ≥ 128 bits; CBC or GCM mode).
- Asymmetric algorithm: RSA (key size ≥ 2048 bits; OAEP or PKCS#1 v1.5 padding).
- Implementation language: Python (PyCryptodome library) or equivalent open-source tooling.
- Private keys must remain confidential; public keys may be shared freely.
- Fresh session keys and correct IV/nonce must be used for each encryption run.

E. Student Work

1. Problem Understanding and Formulation

Clearly restate the enterprise encryption problem in your own words. Identify:

- The specific symmetric, asymmetric, and hybrid design challenges implied by the scenario.
- Expected outcomes: successful hybrid encryption, session-key protection, decryption and verification.
- Assumptions you make (e.g., plaintext type and size, RSA key size, AES key size, mode of operation) — state and justify all assumed values.
- Constraints from Section D that must be satisfied by your design.

2. Application of Course Knowledge (CO3–CO5)

Part I – Symmetric and Asymmetric Cryptography

1. Explain the AES algorithm (key schedule, SubBytes, ShiftRows, MixColumns, AddRoundKey) and RSA algorithm (key generation, encryption, decryption using modular exponentiation). For each, state the data structures and parameters required.
2. Given representative plaintext (e.g., a 512-byte confidential memo), show step-by-step how AES encrypts the data and how RSA protects the resulting session key.
3. Derive or estimate the computational cost (number of operations) of AES-encrypting the plaintext versus RSA-encrypting the same plaintext, and use this to justify the hybrid design decision.

Part II – Hybrid Framework Design and Security Analysis

4. Design the complete hybrid communication workflow: key generation → AES encryption → RSA session-key protection → transfer → RSA session-key recovery → AES decryption → verification. Present this as a numbered algorithm and a flowchart.
5. For the representative plaintext and a given RSA key size, calculate or estimate: AES encryption time, RSA key-wrapping time, RSA key-unwrapping time, and total hybrid processing time. Present results in a table and compare with symmetric-only and asymmetric-only timings.
6. Analyse the security properties: confidentiality, integrity (noting the limitation of unauthenticated modes), key-management complexity, and scalability for multiple users and geographically distributed offices.

Part III – Evaluation and Comparison

7. Compare the three approaches (symmetric-only, asymmetric-only, hybrid AES+RSA) with respect to bulk-encryption efficiency, key distribution, large-data suitability, key-management complexity, computational overhead, and scalability.
8. Discuss the major trade-off identified by the assignment: computational efficiency versus secure key management. Justify your recommendation of the hybrid architecture with reference to Section D.
9. Relate your recommended framework to how a real enterprise PKI (certificate authority, certificate revocation, key rotation) would support and extend it.

3. Solution / Design / Methodology

Integrate your Part I–III analyses into a single hybrid encryption framework design. Your design (and implementation in Section 4) should:

- Accept a plaintext message representing enterprise data.
- Generate or establish an AES session key and an RSA key pair.

- Encrypt the plaintext using AES.
- Protect the AES session key using the recipient's RSA public key.
- Transfer the encrypted data and protected session key.
- Recover the session key using the RSA private key and decrypt the ciphertext.
- Verify that the recovered plaintext matches the original.
- Report encryption and decryption times, key sizes, and ciphertext lengths.

Also present a simple architecture diagram showing the information flow from plaintext through AES encryption, RSA session-key wrapping, transfer, RSA unwrapping, and AES decryption.

4. Use of Modern Tools

Implement and evidence your design using appropriate tools, for example:

- Python / PyCryptodome: implement the hybrid encryption framework covering AES (CBC or GCM) for data encryption and RSA (OAEP or PKCS#1 v1.5) for session-key protection; include source code and console/output screenshots.
- Run at least two workload scenarios (e.g., a short 100-byte message and a long 10,000-byte document) and capture the resulting ciphertext lengths, key sizes, processing times and verification outcomes.
- Any cryptographic visualizer or online tool (e.g., CyberChef) may additionally be used to cross-verify manual calculations.

5. Results and Validation

Present:

- Ciphertext output and recovered plaintext for each test scenario.
- AES encryption time, RSA key-wrapping time, RSA key-unwrapping time, and total hybrid time.
- Comparison table of symmetric-only vs. asymmetric-only vs. hybrid timings.
- Test-case results (see Section 4 below for the test-case table).
- Validation that the system correctly rejects decryption with a wrong key and detects modified ciphertext (where authenticated encryption is used).

6. Analysis and Engineering Decision

- Compare symmetric-only, asymmetric-only, and hybrid approaches and justify which best suits enterprise communication requirements.
- Discuss the major trade-off: computational efficiency versus secure key management.
- Discuss trade-offs of the chosen hybrid strategy versus the alternatives.
- Justify the overall integrated design using quantitative results from Section 5.

7. Broader Considerations

- Security & Privacy: How does your design ensure confidentiality and, where applicable, integrity of enterprise messages?
- Sustainability / Economics: How does hybrid encryption reduce computational cost compared with asymmetric-only bulk encryption, and how does this affect operational energy and infrastructure cost?
- Accessibility & Society: How does the design ensure secure, fair access for all authorized users regardless of their location or device?

- Ethics / Professional Responsibility: Considerations in protecting user data, managing private-key access, and avoiding misuse of cryptographic authority.

8. Conclusion

Summarize your proposed hybrid encryption framework for enterprise networks. State the major findings from your comparative analysis, confirm which stated requirements (Section D) are satisfied, and note any limitations and possible future improvements (e.g., authenticated encryption modes, PKI integration, hardware security modules, adaptive key rotation).

9. Student Reflection

- What did you learn from implementing a hybrid cryptographic framework that combines AES and RSA, beyond what was covered separately in class?
- If given additional time/resources, what would you improve in your design (e.g., replacing CBC with AES-GCM for authenticated encryption, integrating a certificate authority, measuring performance on real enterprise hardware), and why?

10. References

Cite all textbooks, standards, software/simulation tools, and any AI-assisted tools used, in a consistent citation format (e.g., IEEE).

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge (CO3–CO5)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Task	CO	PO	Bloom's Level	Marks
Problem formulation	CO3, CO4	PO1, PO2	L3	10
Application of knowledge – Symmetric & Asymmetric cryptography	CO3	PO1, PO2, PO3	L3/L4	15
Application of knowledge – Hybrid framework design & analysis	CO4	PO2, PO3	L3/L4	15
Solution / Design – Hybrid encryption implementation	CO4, CO5	PO2, PO3, PO5	L4/L5	15
Modern tool usage (Python / PyCryptodome)	CO4	PO5	L3	10
Validation of results	CO5	PO4	L4/L5	15
Analysis & justification / trade-offs	CO4, CO5	PO2, PO4	L4/L5	10

Assessment Task	CO	PO	Bloom's Level	Marks
Broader considerations & documentation/reflection	CO5	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA5116 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment	Threshold
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

1. PROBLEM STATEMENT

The supplied assignment document defines this task as "Design and Evaluation of a Secure Symmetric-Asymmetric Encryption Framework for Enterprise Networks." The assignment describes a modern enterprise environment in which large volumes of sensitive information must be protected from unauthorized access, interception, data leakage and cyberattacks. It explicitly identifies the limitations of relying on a single cryptographic family.

With symmetric-only encryption, a shared secret key is used to protect data efficiently. This provides strong processing efficiency for large messages, but secure key distribution becomes difficult when communication involves many users and geographically distributed offices.

With asymmetric-only encryption, public and private keys make secure key distribution more practical. However, asymmetric operations require substantially more computational resources and are inefficient when applied directly to large volumes of enterprise data.

The proposed response is a hybrid encryption architecture. AES is selected for bulk data encryption and RSA is selected for secure protection and exchange of the symmetric session key.

1.1 Problem Analysis

Issue	Symmetric Only	Asymmetric Only	Hybrid Response
Bulk data processing	Efficient	Computationally expensive	Use symmetric encryption (AES)
Key distribution	Major challenge	More manageable via public keys	Protect session key asymmetrically (RSA)
Enterprise scalability	Key-management burden grows	High processing overhead	Short-lived session keys reduce exposure
Confidentiality	Strong when keys are protected	Strong when implemented correctly	Combines efficient data protection with protected key transfer
Operational complexity	Key lifecycle difficult	Key/certificate management required	Requires both systems but separates their roles

1.2 Expected Outcome

The expected outcome is a secure and scalable framework in which the plaintext is encrypted with a symmetric session key, the session key is protected using the recipient's public key, the encrypted data and protected session key are transferred, and the recipient uses the corresponding private key to recover the session key before decrypting the data. The assignment further expects a comparison against symmetric-only and asymmetric-only approaches.

2. OBJECTIVE

The primary objective is to design and evaluate a secure symmetric–asymmetric hybrid encryption framework for enterprise networks. The objectives below are derived from the assignment brief.

- Design a hybrid encryption architecture combining the efficiency of symmetric cryptography with the key-management advantages of asymmetric cryptography.
- Use AES as the bulk-data encryption mechanism and RSA for protecting and exchanging the symmetric session key.
- Generate or establish the cryptographic keys required for the communication process.
- Encrypt representative plaintext using a symmetric session key.

- Protect the session key using the recipient's RSA public key.
- Demonstrate secure transfer of the encrypted data and protected session key.
- Recover the session key using the RSA private key and decrypt the original plaintext.
- Analyse key generation, encryption, decryption, computational overhead and key-management requirements.
- Compare the hybrid approach with symmetric-only and asymmetric-only approaches.
- Evaluate confidentiality, processing efficiency, scalability, reliability, security strength and key-management complexity.
- Identify the major security/performance trade-off and justify the final architectural choice.
- Recommend a practical deployment strategy for secure enterprise network communication.

2.1 Scope

The scope covers a representative enterprise communication workflow rather than a production deployment. Production-grade certificate authorities, hardware security modules, enterprise identity systems, secure key rotation policies and high-availability infrastructure are outside the demonstrative scope but are discussed as possible extensions.

3. REQUIREMENTS AND ENVIRONMENT USED

3.1 Functional Requirements

1. The system shall accept a plaintext message or representative confidential document.
2. It shall generate or establish a symmetric session key for the communication session.
3. It shall encrypt the plaintext using the symmetric mechanism (AES).
4. It shall protect the session key using the recipient's asymmetric public key (RSA).
5. It shall represent the encrypted message and protected key as transferable ciphertext information.
6. It shall recover the session key using the authorized recipient's private key.
7. It shall decrypt the ciphertext and recover the original plaintext.
8. It shall verify that the recovered plaintext matches the original plaintext.
9. It shall support test cases for valid and invalid decryption conditions.
10. It shall provide evidence of each major processing stage.

3.2 Non-Functional Requirements

Requirement	Description
Security	Keys and plaintext should be handled only for the required operation and not unnecessarily exposed.
Confidentiality	Intercepted ciphertext should not directly reveal the original message.
Performance	Bulk data encryption should be handled by the efficient symmetric mechanism.
Scalability	The architecture should suit multiple users and distributed offices with proper key management.
Reliability	Correct key use should reproduce the original plaintext without alteration.
Maintainability	The system should separate data encryption from session-key protection.
Usability	The workflow should be understandable and repeatable for authorized users.
Auditability	Important security operations should be observable through controlled logs or output messages.

3.3 Environment

- Language: Python 3.x
- Library: PyCryptodome (pip install pycryptodome)
- AES: 256-bit key, CBC mode with PKCS#7 padding (or GCM for authenticated encryption)
- RSA: 2048-bit key pair, OAEP padding with SHA-256
- Platform: Any OS with Python 3 and PyCryptodome installed

4. DESIGN / PROPOSED SOLUTION

The hybrid encryption framework assigns each cryptographic mechanism the task it is best suited for: AES handles efficient bulk-data encryption and RSA handles secure session-key protection and transfer.

4.1 High-Level Architecture

Sender side:

- Generate a random AES-256 session key and IV for each session.
- Encrypt the plaintext using AES-CBC (or AES-GCM) with the session key → produce AES ciphertext.
- Encrypt the session key using the recipient's RSA-2048 public key (OAEP padding) → produce RSA-wrapped key.
- Transmit the bundle: { IV || AES ciphertext || RSA-wrapped session key }.

Receiver side:

- Decrypt the RSA-wrapped session key using the RSA private key → recover AES session key.
- Decrypt the AES ciphertext using the recovered session key and IV → recover original plaintext.
- Verify round-trip equality: recovered plaintext == original plaintext.

4.2 Data Structures

Component	Description
RSA Public Key	2048-bit public modulus and exponent; used by sender to wrap session key
RSA Private Key	2048-bit private key; held only by recipient; used to unwrap session key
AES Session Key	256-bit random bytes; used to encrypt and decrypt plaintext
IV (Initialization Vector)	16-byte random value; prepended to AES ciphertext; ensures different ciphertexts for identical plaintexts
AES Ciphertext	Encrypted plaintext; variable length; padded to AES block boundary
RSA-Wrapped Key	256-byte (2048-bit RSA) blob containing the encrypted session key

5. ALGORITHM, PSEUDOCODE AND FLOWCHART

5.1 Hybrid Encryption Algorithm

Algorithm HYBRID_ENCRYPT(plaintext, recipient_public_key):

1. session_key ← RANDOM_BYTES(32) // 256-bit AES key
2. iv ← RANDOM_BYTES(16) // AES-CBC IV
3. aes_cipher ← AES_CBC_INIT(session_key, iv)
4. padded_plain ← PKCS7_PAD(plaintext)
5. aes_ciphertext ← aes_cipher.ENCRYPT(padded_plain)
6. rsa_cipher ← RSA_OAEP_INIT(recipient_public_key)

7. wrapped_key $\leftarrow$ rsa_cipher.ENCRYPT(session_key)
8. RETURN { iv, aes_ciphertext, wrapped_key }

Algorithm HYBRID_DECRYPT(bundle, recipient_private_key):

1. { iv, aes_ciphertext, wrapped_key } $\leftarrow$ bundle
2. rsa_cipher $\leftarrow$ RSA_OAEP_INIT(recipient_private_key)
3. session_key $\leftarrow$ rsa_cipher.DECRYPT(wrapped_key)
4. aes_cipher $\leftarrow$ AES_CBC_INIT(session_key, iv)
5. padded_plain $\leftarrow$ aes_cipher.DECRYPT(aes_ciphertext)
6. plaintext $\leftarrow$ PKCS7_UNPAD(padded_plain)
7. RETURN plaintext

6. IMPLEMENTATION / SOURCE CODE

The Python implementation below uses PyCryptodome to demonstrate the complete hybrid encryption and decryption cycle.

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import time

# — Key Generation
rsa_key = RSA.generate(2048)
pub_key = rsa_key.publickey()

# — Plaintext
plaintext = b"Confidential enterprise message: Q3 financial report summary."

# — Hybrid Encryption
t0 = time.perf_counter()
session_key = get_random_bytes(32)  # 256-bit AES key
iv = get_random_bytes(16)  # AES-CBC
aes_enc = AES.new(session_key, AES.MODE_CBC, iv)
ciphertext = aes_enc.encrypt(pad(plaintext, AES.block_size))
rsa_enc = PKCS1_OAEP.new(pub_key)
wrapped_key = rsa_enc.encrypt(session_key)
t_enc = time.perf_counter() - t0

# — Hybrid Decryption
t1 = time.perf_counter()
rsa_dec = PKCS1_OAEP.new(rsa_key)
session_key2 = rsa_dec.decrypt(wrapped_key)
aes_dec = AES.new(session_key2, AES.MODE_CBC, iv)
recovered = unpad(aes_dec.decrypt(ciphertext), AES.block_size)
t_dec = time.perf_counter() - t1

# — Verification
print("Plaintext      :", plaintext.decode())
print("Ciphertext length:", len(ciphertext), "bytes")
print("Wrapped key len   :", len(wrapped_key), "bytes")
print("Recovered          :", recovered.decode())
print("Verification      :", "SUCCESS" if recovered == plaintext else "FAILED")
print(f"Enc time : {t_enc*1000:.3f} ms")
print(f"Dec time : {t_dec*1000:.3f} ms")
```

7. TEST CASES AND EXPECTED / ACTUAL RESULTS

TC ID	Test Case	Input	Expected Result	Actual Result
TC-01	Valid encryption and decryption	Input plaintext, valid RSA key pair, valid AES session key	Recovered plaintext equals original plaintext	SUCCESS / FAILED
TC-02	Wrong RSA private key for decryption	Use a different RSA private key than the one corresponding to the public key used	Session-key recovery fails; decryption cannot proceed	SUCCESS / FAILED
TC-03	Modified ciphertext integrity check	Alter one byte of the AES ciphertext before decryption	Decrypted output differs from original or decryption error raised	SUCCESS / FAILED
TC-04	Empty plaintext	Supply an empty string as input	System handles gracefully; no crash; output is empty or equivalent	SUCCESS / FAILED
TC-05	Large plaintext	Supply a plaintext exceeding 10,000 characters	Encryption and decryption complete correctly within acceptable time	SUCCESS / FAILED

8. EXECUTION SCREENSHOTS / OUTPUT

Output Field	Observed Value
Original plaintext	Confidential enterprise message: Q3 financial report summary.
AES session key status	Generated / protected
AES ciphertext length	_____
RSA-protected session-key length	_____
Recovered plaintext	_____
Verification	SUCCESS / FAILED
AES encryption time	_____ ms
RSA key-protection time	_____ ms
Total hybrid processing time	_____ ms

9. ANALYSIS AND DISCUSSION

9.1 Security Strength

The hybrid architecture improves the overall design by assigning different security responsibilities to different mechanisms. AES protects the message content, while RSA protects the session key. The security of the overall system depends on correct implementation and key management; combining algorithms does not automatically make a system secure. A major advantage is that the secret session key is not transmitted as plaintext — it is protected using the recipient's public key, while the corresponding private key is retained by the recipient.

9.2 Confidentiality

Confidentiality is obtained by encrypting the plaintext before transmission. An observer of the communication channel sees encrypted information rather than the original message. The session key provides the secret needed to recover the data, and its protected form prevents direct disclosure during transfer.

9.3 Key-Management Complexity

Key management is the most important operational challenge in the proposed architecture. The enterprise must manage long-term RSA keys and short-lived session keys. Public keys need to be correctly associated with

recipients, while private keys must remain confidential. Session keys should be generated securely and should not be reused unnecessarily.

9.4 Scalability

A hybrid model is more suitable than encrypting every large document directly with RSA because the bulk operation remains symmetric. The architecture can therefore scale more naturally with increasing data volume, provided that identity, public-key distribution, private-key protection and session-key lifecycle processes are properly designed.

9.5 Reliability

For a valid key pair and unchanged ciphertext, the receiver should recover the exact original plaintext. Reliability also depends on correct IV/nonce handling, correct padding, and secure error handling. In production, authenticated encryption such as AES-GCM should be used so that unauthorized modification is detected.

9.6 Comparison of Approaches

Evaluation Factor	Symmetric Only	Asymmetric Only	Hybrid AES + RSA
Bulk encryption efficiency	High	Low	High
Key distribution	Difficult	More manageable	More manageable for session key
Large-data suitability	Excellent	Poor	Excellent
Key-management complexity	High as participants grow	High	Moderate to high
Computational overhead	Low	High	Balanced
Scalability for enterprise data	Good if keys managed well	Limited by computational cost	Strong candidate
Primary strength	Speed	Public/private key model	Combines speed and protected key transfer
Primary limitation	Shared-secret distribution	Expensive for bulk data	More components and lifecycle management

9.7 Major Trade-Off Identified

The major trade-off is between computational efficiency and secure key management. Symmetric encryption is efficient but creates a key-distribution problem. Asymmetric encryption simplifies key distribution but is computationally expensive for large data. The hybrid architecture accepts additional design and management complexity in order to obtain both efficient bulk encryption and protected session-key transfer.

9.8 Advantages

- Efficient encryption of large messages using a symmetric mechanism.
- Protected transfer of the symmetric session key using an asymmetric mechanism.
- Separation of data protection and key protection responsibilities.
- Better fit for distributed enterprise communication than manually sharing a bulk secret.
- Reduced asymmetric workload because only the small session key is protected asymmetrically.

9.9 Limitations

- Requires management of both symmetric and asymmetric keys.

- Incorrect public-key association can create impersonation or key-substitution risks.
- Private-key compromise can expose protected session keys.
- Basic unauthenticated CBC mode does not by itself guarantee integrity — AES-GCM should be used in production.
- Actual performance depends on hardware, implementation, library and message size.

9.10 Security Improvements for Production

- Use AES-GCM or another authenticated-encryption construction instead of unauthenticated CBC.
- Use RSA-OAEP with SHA-256 (or an approved modern equivalent) for RSA key-wrapping.
- Validate recipient public keys using a trusted PKI / certificate process.
- Protect private keys using strong access controls and hardware-backed key storage where justified.
- Use fresh session keys and correct nonce/IV for each encryption run.
- Define key rotation, revocation, backup and recovery procedures.

10. BROADER CONSIDERATIONS

10.1 Security and Privacy

The framework is directly relevant to privacy because enterprise messages may contain customer, employee and financial information. Encryption reduces exposure when communication crosses an untrusted network.

Nevertheless, encryption must be combined with access control, endpoint security, secure authentication and appropriate data-handling practices.

10.2 Professional Responsibility

Cryptographic software should not be presented as secure merely because it successfully encrypts and decrypts a message. A professional security analyst must understand threat models, implementation limitations, key lifecycle requirements and the difference between confidentiality and authenticated integrity.

10.3 Economic and Operational Considerations

The hybrid architecture may increase initial development and key-management effort. However, using asymmetric cryptography only where needed can reduce computational cost compared with asymmetric-only bulk encryption — a balance that is valuable for enterprise systems where both performance and secure key management matter.

10.4 Sustainability

Efficient cryptographic processing can reduce unnecessary computational work. A design that uses the relatively efficient symmetric mechanism for bulk data and limits expensive asymmetric operations to session-key protection can be more resource-conscious and energy-efficient than an asymmetric-only design.

11. CONCLUSION

This assignment analysed the design of a secure symmetric–asymmetric encryption framework for enterprise networks. The problem is a practical conflict between the efficiency of symmetric cryptography and the key-distribution advantages of asymmetric cryptography. A hybrid architecture addresses this conflict by using AES for bulk data and RSA for protection and transfer of the AES session key.

The proposed workflow generates the required keys, encrypts the plaintext using a symmetric session key, protects that session key using the recipient's public key, transfers the protected package, recovers the session key using the private key and decrypts the original message. Functional validation is based on successful recovery of the original plaintext, while negative tests evaluate wrong-key and modified-data conditions.

The analysis confirms that the hybrid approach introduces additional key-management and implementation complexity, but this is the major trade-off accepted to achieve both performance and practical key protection. For real-world deployment, the educational implementation should be strengthened with authenticated encryption, robust public-key validation, secure private-key storage, key rotation and revocation, and audited cryptographic libraries.

12. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Member	Register No.	Contribution	Evidence
Atchaya Chandran R	192521394	Problem analysis, objectives and requirements	Draft sections / notes

13. STUDENT REFLECTION – QUESTION 2 ONLY

Question 2: What major trade-off was identified?

The major trade-off identified in this assignment was between computational efficiency and secure key management. Symmetric encryption is highly efficient for protecting large amounts of data, but it requires the communicating parties to possess the same secret key. In an enterprise with many users and geographically distributed offices, securely distributing and managing these shared keys becomes difficult.

Asymmetric encryption addresses the key-distribution problem more effectively because a public key can be used to protect information intended for the corresponding private-key holder. However, asymmetric cryptographic operations require significantly more computational resources and are therefore inefficient when used directly on large enterprise documents or high-volume data streams.

The hybrid framework accepts an additional level of implementation and key-management complexity to obtain the advantages of both approaches. AES is used for bulk data because it is efficient, while RSA is used to protect the relatively small AES session key. The major trade-off is therefore not simply 'security versus speed'; it is the balance among security, computational performance, scalability and operational key-management complexity.

This trade-off also showed that selecting a cryptographic mechanism should be based on the complete communication architecture rather than on one algorithm considered in isolation. A secure system must consider how keys are generated, transferred, stored, validated, rotated and revoked, in addition to how plaintext is encrypted.

14. REFERENCES

- [1] CSA5116 – Cryptography and Network Security, Assignment 2: Design and Evaluation of a Secure Symmetric-Asymmetric Encryption Framework for Enterprise Networks.
- [2] National Institute of Standards and Technology (NIST), Advanced Encryption Standard (AES), Federal Information Processing Standards Publication 197.
- [3] National Institute of Standards and Technology (NIST), Recommendation for Key Management: Part 1 – General, NIST Special Publication 800-57.
- [4] Internet Engineering Task Force (IETF), PKCS #1: RSA Cryptography Specifications, RFC 8017.
- [5] Python Software Foundation, Python 3 Documentation. <https://docs.python.org/3/>
- [6] PyCryptodome Project, Cryptographic library for Python. <https://pycryptodome.readthedocs.io/>
- [7] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Pearson, 2022.